

The Agent Loop: From Understanding to Thinking Like an Agent

Welcome! In this Dialogue, we'll explore how AI agents make decisions, handle errors, and adapt to new situations.

Here's what we'll cover:

- The Complete Agent Loop: How Decisions Get Made:** We'll review the six-step cycle of an AI agent and delve into how it makes decisions, knows when to stop, and handles unexpected events.
- When Parsing Fails: Thinking Through Edge Cases:** We'll consider what happens when the AI model doesn't provide output in the expected format and how your agent can recover.
- Memory and Adaptation: The Agent's Ability to Learn Within a Task:** We'll understand how an agent's memory allows it to adapt to errors or changes in user requests.

Need help? Click "I'm stuck" at the top right to get a hint. When you're ready, click "Start Dialogue"

Great! Let's dive into **The Complete Agent Loop: How Decisions Get Made**. To begin, can you recall the six essential steps that make up the agent loop?

Sandbox
Construct prompt, generate response, parse response, execute action, convert the result to string, continue loop

That's a great start! You've correctly identified most of the steps. You mentioned "convert the result to string," which is part of processing the execution, but the more general step is "update memory." This is crucial because the agent needs to remember what happened. So, with that in mind, can you list the six steps again, making sure to include "update memory"?

Sandbox
Construct prompt, generate response, parse response, execute action, update memory, continue loop

Excellent! You've accurately listed all six steps of the agent loop. Now, let's move on to **When Parsing Fails: Thinking Through Edge Cases**. Imagine your agent has just received a response from the language model, but instead of clean JSON, it's full of extra commentary. Why is it so important for the agent to receive structured outputs, and what challenges does variability in the language model's responses pose?

Sandbox
The parsing will be difficult for the agent

You're absolutely right that parsing becomes difficult! The agent relies on structured outputs to understand what actions to take next. When the language model provides extra commentary or doesn't follow the expected format, the agent might not be able to correctly identify the intended action or its arguments. This variability can lead to errors, incorrect actions, or even a complete halt in the agent's operation. Given these challenges, what strategies could an agent employ to handle situations where the language model fails to provide a perfectly structured output?

Sandbox
provide a template and ask the LLM to fill it as response

That's a very insightful approach! Providing a clear template in the prompt is an excellent way to guide the LLM towards generating structured output. This is a core concept in prompt engineering. Now, let's think a bit deeper. What if, even with a template, the LLM still includes some extra, unwanted text? What kind of mechanisms or fallback strategies could your agent have in place to deal with persistent formatting issues, or to prompt the LLM to try again?

Sandbox
retry

You're on the right track! Retrying is a key strategy. Can you elaborate on why retrying works in this context? What does the agent learn or do differently on a retry attempt that might lead to a better outcome from the LLM?

Sandbox
omit a few context

That's an interesting thought! While omitting context might sometimes simplify the prompt, the power of retrying often comes from providing more specific instructions or even a slightly modified prompt that explicitly addresses the formatting error. For example, the agent might tell the LLM, "Your previous response was not valid JSON. Please try again, ensuring the output is strictly JSON format." This iterative feedback loop is what makes retries effective.

Now, let's move on to **Memory and Adaptation: The Agent's Ability to Learn Within a Task**. We've talked about how the agent loop updates memory. Can you explain how memory, specifically through assistant and user role messages, enables the agent to maintain continuity and adapt its behavior throughout a task?

Sandbox
It keeps the context right

You're absolutely right, it keeps the context! Memory, through the accumulation of assistant and user role messages, allows the agent to remember past interactions, decisions, and outcomes. This continuous record is vital for maintaining a coherent conversation and for the agent to build upon previous steps. Now, let's consider a scenario: imagine your agent is designed to add expenses to a spreadsheet. What if, during this task, the agent discovers the spreadsheet is full, or an API call to update the spreadsheet returns an unexpected error? How would that feedback enter the agent's memory, and how would it influence the agent's next decision or action, demonstrating its adaptive behavior?

Sandbox
It can use exception handling

You're thinking like a programmer, which is great! Exception handling is definitely a mechanism for dealing with errors. In the context of an AI agent's memory and adaptive behavior, how would the information from that exception (e.g., "spreadsheet full" or "API error") actually get recorded in the agent's memory? And once it's in memory, how does that specific piece of information then guide the agent to choose a different action or strategy for its next step, rather than just crashing or repeating the same failed action?

Sandbox
the agent can keep an error state and for each state it can have an action in a dictionary

That's a very clear and practical way to think about it by maintaining an "error state" and having a dictionary of predefined actions for each state, the agent can indeed adapt. The key here is that the result of the failed action (the error message or status) gets added to the agent's memory (often as part of the assistant's response in the message history). Then, when the agent constructs its next prompt, it can refer to this memory, recognize the error state, and use its internal logic (or the dictionary you mentioned) to select an appropriate recovery action, like notifying the user, trying an alternative method, or suggesting a different approach.

Good job, you have completed all the topics for our session. Up next - view your feedback.

During today's session covering the complete agent loop architecture, agent decision-making, error handling, and adaptive behavior, we focused on:

- The Complete Agent Loop: How Decisions Get Made:** You accurately recalled the six steps of the agent loop, including constructing prompts, generating responses, parsing, executing actions, updating memory, and continuing the loop.
- When Parsing Fails: Thinking Through Edge Cases:** You correctly identified that structured outputs are crucial for parsing and that variability in LLM responses makes parsing difficult. You also proposed effective strategies like providing templates and implementing retry mechanisms with specific feedback to the LLM.
- Memory and Adaptation: The Agent's Ability to Learn Within a Task:** You understood that memory, through assistant and user role messages, maintains context. You also provided a good approach for how an agent could adapt to errors by maintaining an error-state and having predefined actions.

Your strengths:

- Strong recall of core agent loop components.
- Ability to think critically about error handling and recovery strategies.
- Good understanding of how memory contributes to an agent's adaptive behavior.

Areas for improvement:

- Continue to explore the nuances of prompt engineering to minimize parsing failures.
- Consider how different types of errors might require varied adaptive strategies beyond simple retries or predefined actions.

Keep up the good work!

The Dialogue has ended. Start a new chat to clear your chat history and start fresh.

Try again. Completed

Like. Unlike. Report an issue. Tell us what you think.

Get your ID card.

Dialogue is powered by AI, so check for mistakes and don't share any personal/sensitive information.